

Green Bonds EDA

Noah Leibowitz & Siya Motwani

2026-06-22

```
# Load in all libraries
library(dplyr)
library(tidyverse)
library(readxl)
library(ggplot2)
library(maps)
library(tidyr)

# Solar Data
solar_2024 <- read_excel("Solar.xlsx", sheet = "2024")
solar_2023 <- read_excel("Solar.xlsx", sheet = "2023")
solar_2022 <- read_excel("Solar.xlsx", sheet = "2022")
solar_2021 <- read_excel("Solar.xlsx", sheet = "2021")
solar_2020 <- read_excel("Solar.xlsx", sheet = "2020")
solar_2019 <- read_excel("Solar.xlsx", sheet = "2019")
solar_2018 <- read_excel("Solar.xlsx", sheet = "2018")
solar_2017 <- read_excel("Solar.xlsx", sheet = "2017")
solar_2016 <- read_excel("Solar.xlsx", sheet = "2016")
solar_2015 <- read_excel("Solar.xlsx", sheet = "2015")
solar_2014 <- read_excel("Solar.xlsx", sheet = "2014")
solar_2013 <- read_excel("Solar.xlsx", sheet = "2013")

# Wind Data
wind_2024 <- read_excel("Wind.xlsx", sheet = "2024")
wind_2023 <- read_excel("Wind.xlsx", sheet = "2023")
wind_2022 <- read_excel("Wind.xlsx", sheet = "2022")
wind_2021 <- read_excel("Wind.xlsx", sheet = "2021")
wind_2020 <- read_excel("Wind.xlsx", sheet = "2020")
wind_2019 <- read_excel("Wind.xlsx", sheet = "2019")
wind_2018 <- read_excel("Wind.xlsx", sheet = "2018")
wind_2017 <- read_excel("Wind.xlsx", sheet = "2017")
wind_2016 <- read_excel("Wind.xlsx", sheet = "2016")
wind_2015 <- read_excel("Wind.xlsx", sheet = "2015")
wind_2014 <- read_excel("Wind.xlsx", sheet = "2014")
wind_2013 <- read_excel("Wind.xlsx", sheet = "2013")

# Green Bond Issuance and RPS Data
GBI <- read_excel("GB_RPS Hard Code.xlsx")

# Pre-Trends Wind Data
pre_wind <- read_csv("Wind Pre.csv")
```

```

# Pre-Trends Solar Data
pre_solar <- read_excel("Solar 2010 - 2012.xlsx")

# Cleaning Pre-Wind Data

clean_wind_panel <- pre_wind %>%
# Remove any rows where the operational year is missing/blank
filter(!is.na(p_year)) %>%

# Select only the specific columns your model needs
select(t_state, p_year, t_cap) %>%

# Group by State and Year, and convert individual turbine kW to total MW added
group_by(t_state, p_year) %>%
summarize(mw_added = sum(t_cap, na.rm = TRUE) / 1000, .groups = 'drop') %>%

# Open your 50-state grid for your pre-trends window
complete(t_state, p_year = 2000:2026, fill = list(mw_added = 0)) %>%
arrange(t_state, p_year) %>%
group_by(t_state) %>%

# Cumulative sum holds capacity at 0 until first project,
# then holds total capacity flat forward through the years
mutate(total_wind_mw = cumsum(mw_added)) %>%
rename(state = t_state, year = p_year) %>%

# Filter the outcome variable to strictly include 2010-2012
filter(year >= 2010 & year <= 2012)

# Split by each year and pivot year column to display state and capacity by each year separately
clean_wind_panel <- clean_wind_panel %>%
  select(state, year, total_wind_mw) %>%
  group_by(year) %>%
  group_split(.keep = FALSE)
wind_2010 <- clean_wind_panel[[1]]
wind_2011 <- clean_wind_panel[[2]]
wind_2012 <- clean_wind_panel[[3]]

# Cleaning Pre-Solar Data

# 2. Extract columns by their exact position number instead of text names
solar_wide_cumulative <- pre_solar %>%
  # Select column 2 (State), column 3 (2010), column 4 (2011), and column 5 (2012)
  select(State = 1, 2, 3, 4)

# 3. Check what R actually named these columns so we can use them
col_names <- colnames(solar_wide_cumulative)

# 4. Perform the math using index tracking to prevent text crashes
solar_wide_cumulative <- solar_wide_cumulative %>%
  mutate(
    Year_2010 = .[[2]], # Pulls the data from the 2010 column position
    Year_2011 = .[[2]] + .[[3]], # Adds 2010 + 2011 column positions
  )

```

```

  Year_2012 = (.[[2]] + .[[3]]) + .[[4]] # Adds 2010 + 2011 + 2012 column positions
) %>%

```

```

# 5. Output only your State and the clean, quote-free cumulative totals
select(State, Year_2010, Year_2011, Year_2012)

```

```

# Clean Post-Solar Data

```

```

# Add year columns

```

```

solar_2010 <- solar_wide_cumulative %>%
  select(State, solar_capacity = Year_2010) %>%
  mutate(
    year = 2010,
    State = state.abb[match(State, state.name)]
  ) %>%
  arrange(State)

```

```

solar_2011 <- solar_wide_cumulative %>%
  select(State, solar_capacity = Year_2011) %>%
  mutate(
    year = 2011,
    State = state.abb[match(State, state.name)]) %>%
  arrange(State)

```

```

solar_2012 <- solar_wide_cumulative %>%
  select(State, solar_capacity = Year_2012) %>%
  mutate(
    year = 2012,
    State = state.abb[match(State, state.name)]) %>%
  arrange(State)

```

```

# Loop through every year from 2013 to 2024

```

```

for (y in 2013:2024) {

```

```

  # 1. Dynamically get the variable name (e.g., "solar_2013")
  var_name <- paste0("solar_", y)

```

```

  # 2. Grab the dataframe from your environment
  temp_df <- get(var_name)

```

```

  # 3. Apply the tidyverse pipeline to sum up capacities by state

```

```

  cleaned_df <- temp_df %>%
    mutate(year = y) %>% # Dynamically add the current loop's year
    rename(solar_capacity = `Nameplate Capacity (MW)`) %>%
    group_by(State, year) %>%
    summarize(solar_capacity = sum(solar_capacity, na.rm = TRUE), .groups = "drop") %>%
    arrange(State)

```

```

  # 4. Overwrite the messy dataframe in your environment with the cleaned one
  assign(var_name, cleaned_df)
}

```

```

post_solar_all = bind_rows(solar_2010, solar_2011, solar_2012, solar_2013, solar_2014, solar_2015, solar_2016, solar_2017, solar_2018, solar_2019, solar_2020, solar_2021, solar_2022, solar_2023, solar_2024)

```

```

# 1. Create a master dataframe containing exactly the 48 contiguous states
contiguous_48 <- state.abb[!state.abb %in% c("AK", "HI")]
master_states <- data.frame(State = contiguous_48)

# 2. Group and aggregate FIRST to handle hidden duplicates, then pivot wide
solar_wide <- post_solar_all %>%
  group_by(State, year) %>%
  summarize(solar_capacity = sum(solar_capacity, na.rm = TRUE), .groups = "drop") %>%
  pivot_wider(
    names_from = year,
    values_from = solar_capacity,
    names_prefix = "yr_"
  )

# 3. Left join the master list with your solar data and sort alphabetically
final_solar_panel <- master_states %>%
  left_join(solar_wide, by = "State") %>%
  arrange(State)

# 4. Interpolate missing values cleanly
final_solar_panel_clean <- final_solar_panel %>%
  pivot_longer(
    cols = starts_with("yr_"),
    names_to = "year",
    values_to = "solar_capacity"
  ) %>%
  group_by(State) %>%
  arrange(State, year) %>%
  fill(solar_capacity, .direction = "down") %>%
  mutate(solar_capacity = replace_na(solar_capacity, 0)) %>%
  pivot_wider(
    names_from = year,
    values_from = solar_capacity
  ) %>%
  ungroup()

```

```

# Clean Post-Wind Data

```

```

# Add year columns

```

```

wind_2010 <- wind_2010 %>%
  rename(State = `state`, wind_capacity = `total_wind_mw`) %>%
  mutate(year = 2010) %>%
  arrange(State)

```

```

wind_2011 <- wind_2011 %>%
  rename(State = `state`, wind_capacity = `total_wind_mw`) %>%
  mutate(year = 2011) %>%
  arrange(State)

```

```

wind_2012 <- wind_2012 %>%
  rename(State = `state`, wind_capacity = `total_wind_mw`) %>%
  mutate(year = 2012) %>%
  arrange(State)

```

```

# Loop through every year from 2013 to 2024
for (y in 2013:2024) {

  # 1. Dynamically generate the variable name (e.g., "wind_2013")
  var_name <- paste0("wind_", y)

  # 2. Grab the dataframe from your environment using its text name
  temp_df <- get(var_name)

  # 3. Apply your tidyverse cleaning pipeline
  cleaned_df <- temp_df %>%
    mutate(year = y) %>% # Dynamically adds the current loop's year
    rename(wind_capacity = `Nameplate Capacity (MW)`) %>%
    group_by(State, year) %>%
    summarize(wind_capacity = sum(wind_capacity, na.rm = TRUE), .groups = "drop") %>%
    arrange(State)

  # 4. Save the cleaned dataframe back to its original variable name
  assign(var_name, cleaned_df)
}

post_wind_all = bind_rows(wind_2010, wind_2011, wind_2012, wind_2013, wind_2014, wind_2015, wind_2016, wind_2017, wind_2018, wind_2019, wind_2020, wind_2021, wind_2022, wind_2023, wind_2024)

# 1. Create a master dataframe containing exactly the 48 contiguous states
contiguous_48 <- state.abb[!state.abb %in% c("AK", "HI")]
master_states <- data.frame(State = contiguous_48)

# 2. Pivot your existing combined data to wide format
wind_wide <- post_wind_all %>%
  pivot_wider(
    names_from = year,
    values_from = wind_capacity,
    names_prefix = "yr_"
  )

# 3. Left join the master list with your data and sort alphabetically
final_wind_panel <- master_states %>%
  left_join(wind_wide, by = "State") %>%
  arrange(State)

final_wind_panel_clean <- final_wind_panel %>%
  # 1. Temporarily bring columns back to a long format to work row-by-row per state
  pivot_longer(
    cols = starts_with("yr_"),
    names_to = "year",
    values_to = "wind_capacity"
  ) %>%
  # 2. Group by State and sort by year to ensure sequential order
  group_by(State) %>%
  arrange(State, year) %>%
  # 3. Fill NAs with the previous year's value (Last Observation Carried Forward)
  fill(wind_capacity, .direction = "down") %>%
  # 4. If it's still NA (meaning it never did anything yet), replace with 0
  replace_na(list(wind_capacity = 0))

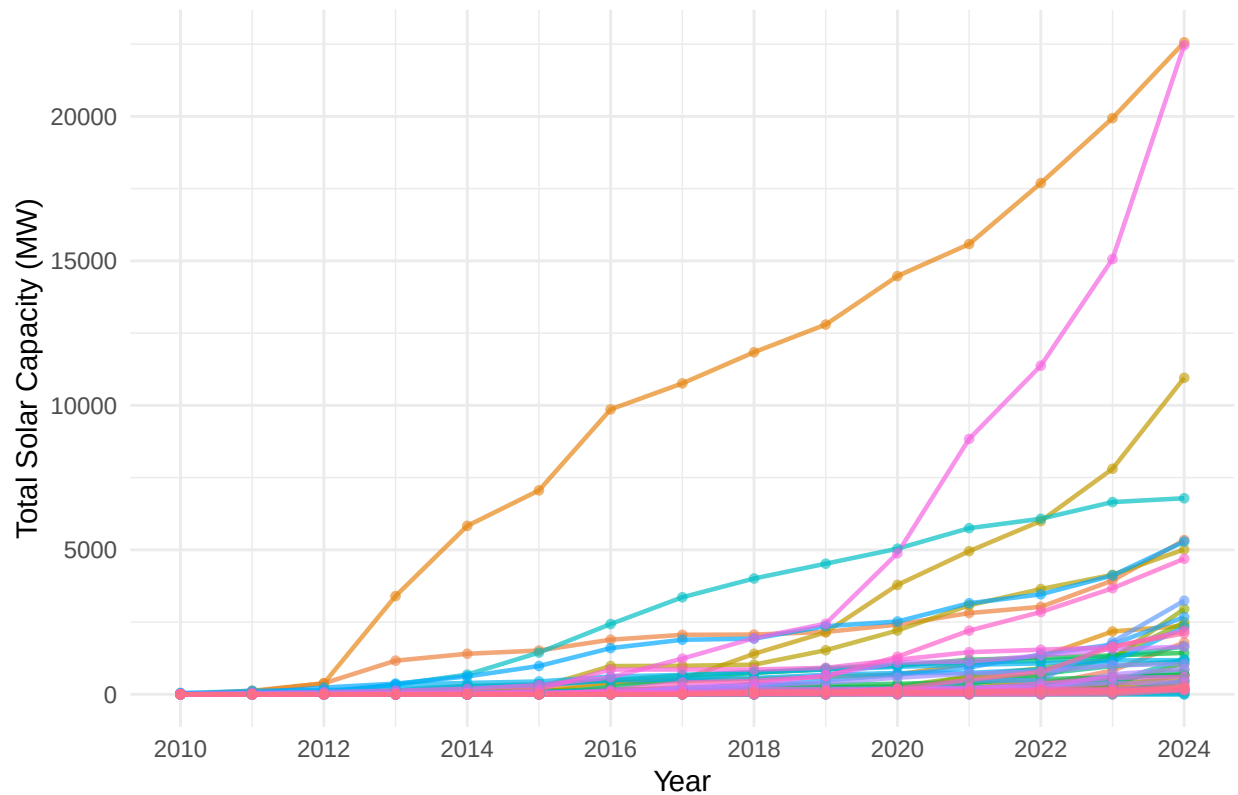
```

```
mutate(wind_capacity = replace_na(wind_capacity, 0)) %>%
# 5. Bring it back to your beautiful wide 48-state layout
pivot_wider(
  names_from = year,
  values_from = wind_capacity
) %>%
ungroup()
```

```
# 1. Transform wide data to long format for all years
solar_plot_data_all <- final_solar_panel_clean %>%
  pivot_longer(
    cols = starts_with("yr_"),
    names_to = "year",
    values_to = "capacity"
  ) %>%
  # Clean up the "yr_" prefix and turn year into a number
  mutate(year = as.numeric(gsub("yr_", "", year)))

# 2. Create the full-timeline line plot
ggplot(solar_plot_data_all, aes(x = year, y = capacity, group = State, color = State)) +
  geom_line(size = 0.8, alpha = 0.7) +
  geom_point(size = 1.2, alpha = 0.6) +
  # Ensure the X-axis handles the wide timeframe cleanly
  scale_x_continuous(breaks = seq(2010, 2024, by = 2)) +
  labs(
    title = "Solar Capacity Over Time by State (2010 - 2024)",
    x = "Year",
    y = "Total Solar Capacity (MW)"
  ) +
  theme_minimal() +
  theme(legend.position = "none") # Hides the messy 48-state legend
```

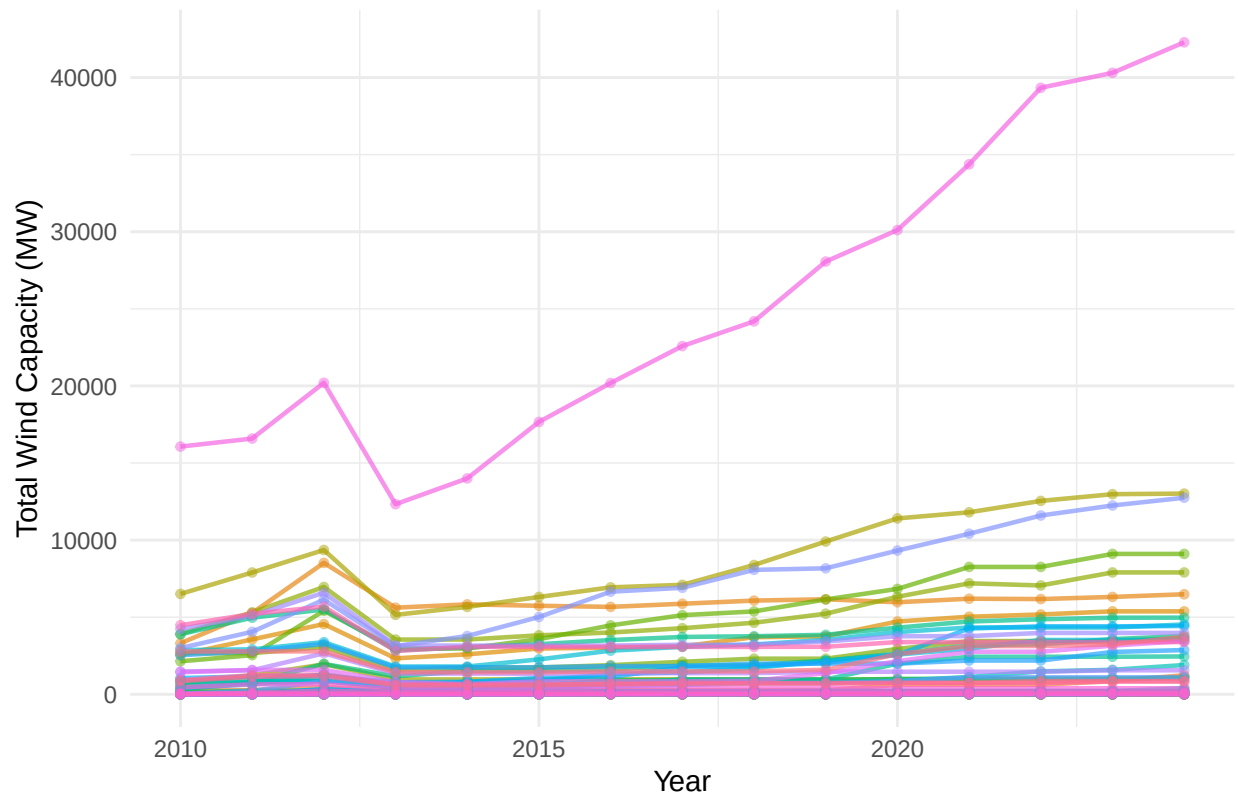
Solar Capacity Over Time by State (2010 – 2024)



```
# 1. Transform wide data to long format for plotting
plot_data <- final_wind_panel_clean %>%
  pivot_longer(
    cols = starts_with("yr_"),
    names_to = "year",
    values_to = "capacity"
  ) %>%
  # Clean up the "yr_" prefix from the year column so it treats years as numbers
  mutate(year = as.numeric(gsub("yr_", "", year)))

# 2. Create the line plot
ggplot(plot_data, aes(x = year, y = capacity, group = State, color = State)) +
  geom_line(size = 0.8, alpha = 0.7) +
  geom_point(size = 1.2, alpha = 0.5) +
  labs(
    title = "Wind Capacity Over Time by State (48 Contiguous)",
    x = "Year",
    y = "Total Wind Capacity (MW)"
  ) +
  theme_minimal() +
  # Since 48 states will make a massive legend, we can hide it
  # or keep it depending on your preference:
  theme(legend.position = "none")
```

Wind Capacity Over Time by State (48 Contiguous)



```
# Prepare 2024 data
wind_map_2024 <- final_wind_panel_clean %>%
  select(State, capacity = yr_2024) %>%
  mutate(region = tolower(state.name[match(State, state.abb)]))

us_states <- map_data("state")

wind_geo_data <- us_states %>%
  left_join(wind_map_2024, by = "region")

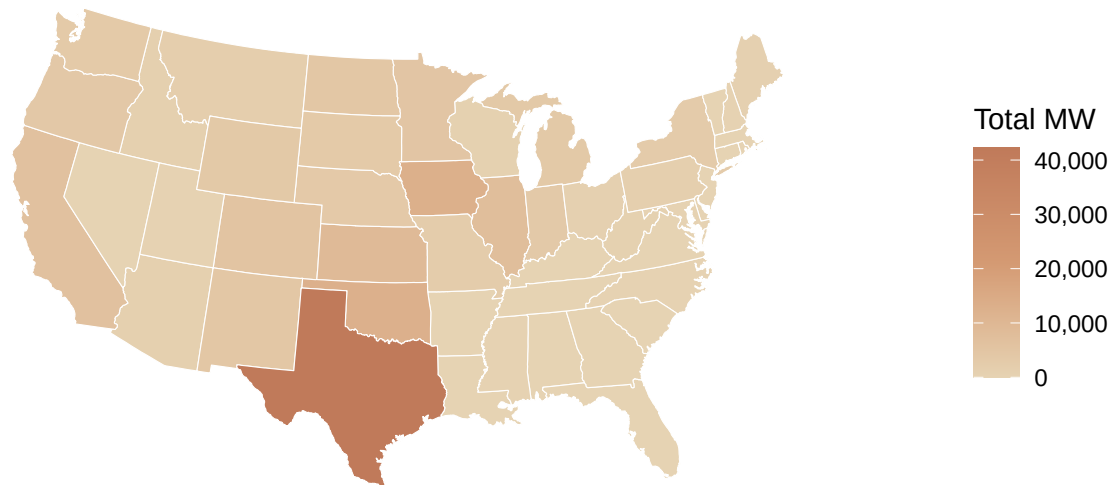
# Generate Wind Map with Custom Palette
ggplot(wind_geo_data, aes(x = long, y = lat, group = group, fill = capacity)) +
  geom_polygon(color = "white", size = 0.2) +
  # Custom continuous gradient interpolation using your colors
  scale_fill_gradientn(
    name = "Total MW",
    colors = c("#E6D3B3", "#D49A73", "#C07A5A"),
    labels = scales::comma_format(),
    na.value = "grey90"
  ) +
  coord_map("albers", lat0 = 39, lat1 = 45) +
  labs(
    title = "Cumulative US Wind Capacity by State (2024)",
    subtitle = "48 Contiguous States"
  ) +
  theme_void() +
```



```
theme(legend.position = "right")
```

Cumulative US Wind Capacity by State (2024)

48 Contiguous States



```
# Prepare 2024 data
solar_map_2024 <- final_solar_panel_clean %>%
  select(State, capacity = yr_2024) %>%
  mutate(region = tolower(state.name[match(State, state.abb)]))

solar_geo_data <- us_states %>%
  left_join(solar_map_2024, by = "region")

# Generate Solar Map with Custom Palette
ggplot(solar_geo_data, aes(x = long, y = lat, group = group, fill = capacity)) +
  geom_polygon(color = "white", size = 0.2) +
  # Custom continuous gradient interpolation using your colors
  scale_fill_gradientn(
    name = "Total MW",
    colors = c("#E6D3B3", "#D49A73", "#C07A5A"),
    labels = scales::comma_format(),
    na.value = "grey90"
  ) +
  coord_map("albers", lat0 = 39, lat1 = 45) +
  labs(
    title = "Cumulative US Solar Capacity by State (2024)",
    subtitle = "48 Contiguous States"
  ) +
```

```
theme_void() +  
theme(legend.position = "right")
```

Cumulative US Solar Capacity by State (2024)

48 Contiguous States

